SUVARNA TECHNOSOFT

Angular + ASP.NET Core Web API + IIS Hosting

Professional Deployment SOP

Team-Share Ready Standard Operating Procedure

Prepared By: Ravi Chenikala

Role: Sr. Project Lead

Audience: Junior Developers / Deployment Team / Support Team

Version: 1.0

Date: 26-Mar-2026

Purpose: This document provides a production-oriented, step-by-step guide to build Angular applications, build/publish ASP.NET Core Web APIs, and host both in IIS with deployment standards, troubleshooting, security, and rollback guidance.

SUVARNA TECHNOSOFT

Table of Contents

Note: In Microsoft Word, right-click this page after opening the document and choose 'Update Field' if you add an automatic TOC later.

- 1. Document Objective & Scope
- 2. Architecture Overview
- 3. Standard Software & Version Matrix
- 4. Angular Build SOP
- 5. ASP.NET Core Web API Build & Publish SOP
- 6. IIS Hosting SOP
- 7. Deployment Models (Same Server / Separate Server / Reverse Proxy)
- 8. Configuration Standards
- 9. Validation & Smoke Test Checklist
- 10. Troubleshooting Matrix
- 11. Security & Production Hardening
- 12. Rollback SOP
- 13. Appendices (Commands / Sample Configs / Naming Standards)

SUVARNA TECHNOSOFT

1. Document Objective & Scope

This SOP is intended to standardize how the team prepares development machines, builds Angular projects, builds and publishes ASP.NET Core Web APIs, deploys them to IIS, validates deployments, troubleshoots failures, and safely rolls back when needed.

Scope

- Angular frontend application setup and production build
- ASP.NET Core Web API setup, restore, build, run, and publish
- IIS prerequisites and application hosting
- Angular hosting in IIS including SPA route handling
- ASP.NET Core API hosting in IIS via ASP.NET Core Module
- CORS, environment configuration, folder permissions, SSL, and health checks
- Deployment verification and rollback strategy

2. Architecture Overview

- Frontend: Angular Application (Static build output)
- Backend: ASP.NET Core Web API
- Web Server: IIS
- Runtime: .NET Runtime / ASP.NET Core Hosting Bundle
- Optional Database: SQL Server / PostgreSQL / MySQL
- Optional Integrations: External APIs, SMTP, SMS, WhatsApp, payment gateways, file storage

Recommended Production Topologies

- Option A: Angular and API hosted as separate IIS sites on same server
- Option B: Angular hosted in IIS, API hosted in separate IIS site/domain/subdomain
- Option C: Angular and API behind reverse proxy / gateway

3. Standard Software & Version Matrix

Always validate project compatibility before installing tools. Do not blindly install latest versions in enterprise projects.

Component	Recommended Standard	Purpose	Notes
Node.js	20.x LTS	Angular build/runtime tooling	Use project-compatible version if package constraints exist
npm	Bundled with Node.js	Package management	Verify with npm -v
Angular CLI	Project compatible	Angular build & serve	Match Angular

	(ex: v18)		workspace version
Visual Studio Code	Latest stable	Frontend editing	Recommended for Angular
.NET SDK	Project target (ex: .NET 8 SDK)	Build / publish API	Must match .csproj target
Visual Studio 2022	Latest stable	API development	Install ASP.NET and web development workload
IIS	Windows IIS	Web hosting	Required for deployment
ASP.NET Core Hosting Bundle	Same major version as target framework	Run ASP.NET Core in IIS	Mandatory
URL Rewrite Module	Latest compatible	SPA routing / redirects	Recommended
Git	Latest stable	Source control	Recommended
Postman / Bruno	Latest stable	API testing	Recommended

Version Verification Commands

```
node -v
npm -v
ng version
dotnet --info
git --version
```

4. Angular Build SOP

4.1 Required Software

- Node.js (LTS, project compatible)
- npm
- Angular CLI
- Visual Studio Code (recommended)
- Git (recommended)
- Chrome / Edge browser

4.2 Recommended VS Code Extensions

- Angular Language Service
- ESLint
- Prettier
- Auto Rename Tag
- Path Intellisense
- GitLens

4.3 Setup Steps

1. Install Node.js LTS
2. Verify Node.js and npm

```
node -v
npm -v
```

3. Install Angular CLI globally (if project requires global CLI usage)

```
npm install -g @angular/cli
```

```
ng version
```

4. Clone repository or copy source code

```
git clone <repository-url>
cd <angular-project-folder>
```

5. Install dependencies

```
npm install
```

6. Validate environment configuration

- Check src/environments/environment.ts
- Check src/environments/environment.prod.ts
- Verify API base URL points to correct environment

7. Run locally for verification

```
ng serve
# or
npm start
```

- Default URL: http://localhost:4200

8. Build for production

```
ng build --configuration production
# or
npm run build
```

4.4 Expected Build Output

- Output folder usually: dist/<project-name>/
- Contains index.html, JS bundles, CSS bundles, assets, runtime chunks

4.5 Angular Production Deployment Checklist

☐Node.js installed and verified

☐Angular CLI installed / compatible

☐npm install successful

☐No dependency vulnerabilities blocking release (review if required)

☐environment.prod.ts verified

☐API URL updated correctly

☐Build completed successfully

☐dist output verified

☐Angular web.config prepared for IIS

4.6 Common Angular Risks / Blind Spots

- Node version mismatch can break builds unexpectedly
- Environment file may still point to local API
- Angular route refresh fails in IIS without rewrite rules
- Secrets must never be stored in Angular environment files
- Hardcoded localhost URLs are a common production mistake

5. ASP.NET Core Web API Build & Publish SOP

5.1 Identify Project Target Framework

Before installing any runtime/SDK, open the .csproj file and confirm the target framework.

```
<TargetFramework>net8.0</TargetFramework>
```

- Examples: net6.0, net7.0, net8.0
- Install exact or compatible SDK version

5.2 Required Software

- .NET SDK (matching project)
- Visual Studio 2022 or VS Code
- ASP.NET Core Runtime
- NuGet package access
- Git

5.3 Setup Steps

9. Install .NET SDK

```
dotnet --info
```

10. Clone repository or open source folder

```
git clone <repository-url>
cd <api-project-folder>
```

11. Restore NuGet packages

```
dotnet restore
```

12. Build the project

```
dotnet build
```

13. Run locally

```
dotnet run
```

- Check launchSettings.json or console output for URLs

14. Validate configuration files

- appsettings.json
- appsettings.Development.json
- appsettings.Production.json (if used)

15. Publish for deployment

```
dotnet publish -c Release -o ./publish
```

5.4 API Production Checklist

☐Correct .NET SDK installed

☐dotnet restore successful

☐dotnet build successful

☐dotnet run successful locally

☐Database connection string validated

☐External API endpoints validated

☐JWT/Auth settings validated

☐CORS configuration validated

☐appsettings.Production.json reviewed

☐dotnet publish successful

☐publish folder generated and reviewed

5.5 Common API Risks / Blind Spots

- Wrong SDK version = build failure or runtime mismatch
- Local DB connection string accidentally deployed to production
- Secrets in appsettings.json instead of secure configuration
- Missing write permission for logs/uploads causes runtime failures
- CORS configured too loosely (AllowAnyOrigin) is a production risk

6. IIS Hosting SOP

6.1 Required IIS Components

- Internet Information Services
- IIS Management Console
- World Wide Web Services
- Common HTTP Features
- Application Development Features
- Security Features
- Performance Features

6.2 Critical Requirement: ASP.NET Core Hosting Bundle

This is mandatory for hosting ASP.NET Core APIs in IIS. Without it, you will hit 500.30 / 500.31 / 502.5 type failures.

- Installs .NET Runtime
- Installs ASP.NET Core Runtime
- Installs ASP.NET Core Module (ANCM) for IIS

6.3 Recommended IIS Add-ons

- URL Rewrite Module (required/recommended for Angular SPA routing)
- Web Deploy (optional)

6.4 Recommended Folder Structure

```
D:\WebApps\
    ├── AngularApp\
    │     ├── index.html
    │     ├── assets\
    │     ├── *.js
    │     ├── *.css
    │     └── web.config
    │
    └── MyApi\
```

```
├── MyApi.dll
├── appsettings.json
├── appsettings.Production.json
├── web.config
├── logs\
└── uploads\
```

6.5 Deployment Copy Rules

- Copy Angular dist/<project-name>/ contents into Angular IIS physical path
- Copy API publish/ contents into API IIS physical path
- Do not copy source code folders unnecessarily to server
- Avoid leaving old DLLs/configs in place during upgrade without cleanup strategy

6.6 Application Pool Standards

Application	App Pool	.NET CLR	Pipeline	Notes
Angular Site	AngularAppPool	No Managed Code	Integrated	Static site in IIS
ASP.NET Core API	MyApiPool	No Managed Code	Integrated	IIS forwards to Kestrel via ANCM

6.7 IIS Site Creation Steps

Angular Site

16. Open IIS Manager
17. Right-click Sites → Add Website
18. Set Site Name (e.g., AngularApp)
19. Set Physical Path (e.g., D:\WebApps\AngularApp\)
20. Configure Binding (HTTP/HTTPS, Port, Host Name)
21. Assign AngularAppPool

API Site

22. Right-click Sites → Add Website (or Add Application)
23. Set Site Name (e.g., MyApi)
24. Set Physical Path (e.g., D:\WebApps\MyApi\)
25. Configure Port / Hostname / HTTPS binding
26. Assign MyApiPool

6.8 Folder Permissions Standard

- Grant Read & Execute to IIS_IUSRS or specific App Pool identity

```
IIS AppPool\AngularAppPool
IIS AppPool\MyApiPool
```

- Grant Modify only to folders that require logs/uploads/temp writes
- Do NOT grant Full Control unless there is a proven operational need

7. Deployment Models

7.1 Model A: Angular + API on Same Server, Different Sites

- Angular: https://app.company.com
- API: https://api.company.com
- Best for clear separation and easier CORS/domain management

7.2 Model B: Angular + API on Same IIS Site (Advanced / Optional)

- Angular hosted as root app
- API hosted as child application, e.g., /api
- Useful when single-domain requirement exists
- Requires careful routing and application path handling

7.3 Model C: Reverse Proxy / Gateway Pattern

- Frontend served from app domain
- API proxied through gateway / reverse proxy
- Reduces browser CORS complexity
- Better for enterprise scaling and centralized security

8. Configuration Standards

8.1 Angular IIS web.config (SPA Routing)

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
<system.webServer>
<rewrite>
<rules>
<rule name="Angular Routes" stopProcessing="true">
<match url=".*" />
<conditions logicalGrouping="MatchAll">
<add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
<add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
</conditions>
<action type="Rewrite" url="/" />
</rule>
</rules>
</rewrite>
<staticContent>
<mimeMap fileExtension=".json" mimeType="application/json" />
```

```
</staticContent>
</system.webServer>
</configuration>
```

8.2 ASP.NET Core web.config (Generated/Verified)

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
<location path="." inheritInChildApplications="false">
<system.webServer>
<handlers>
<add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2"
resourceType="Unspecified" />
</handlers>
<aspNetCore processPath="dotnet" arguments="MyApi.dll"
stdoutLogEnabled="false" stdoutLogFile=".\logs\stdout"
hostingModel="inprocess" />
</system.webServer>
</location>
</configuration>
```

8.3 CORS Standard (Restrictive, Not Open)

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAngularApp", policy =>
    {
        policy.WithOrigins("https://app.company.com")
              .AllowAnyHeader()
              .AllowAnyMethod();
    });
});

app.UseCors("AllowAngularApp");
```

8.4 Sample appsettings.Production.json Structure

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;Database=...;User Id=...;Password=...;"
  },
  "Jwt": {
    "Issuer": "your-issuer",
    "Audience": "your-audience",
    "Key": "DO_NOT_STORE_PLAIN_TEXT_IN_REAL_PROD"
  },
  "AllowedHosts": "*",
  "ExternalApis": {
    "BaseUrl": "https://api.partner.com"
  },
```

```
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

9. Validation & Smoke Test Checklist

9.1 Angular Validation

☐Application loads successfully

☐All JS/CSS assets load (no 404)

☐Browser console has no blocking errors

☐Page refresh works on nested Angular routes

☐Environment points to correct API

9.2 API Validation

☐Site starts successfully in IIS

☐Swagger (if enabled) loads

☐Health endpoint returns success

☐Critical API endpoint returns expected response

☐Database connectivity verified

☐Logs written successfully (if applicable)

9.3 End-to-End Validation

☐Angular can call API successfully

☐No CORS errors in browser

☐Authentication works

☐Uploads/downloads work (if applicable)

☐Error handling pages/messages are acceptable

10. Troubleshooting Matrix

Issue	Likely Cause	Fix / Action
'ng' not recognized	Angular CLI missing	Run npm install -g @angular/cli
Angular build fails	Node version mismatch / package conflict	Verify Node version, delete node_modules, reinstall
Angular refresh gives 404	Missing SPA rewrite rules	Add Angular web.config and ensure URL Rewrite installed
dotnet not recognized	.NET SDK missing	Install correct .NET SDK and reopen terminal
dotnet build fails	NuGet restore issue / wrong SDK	Run dotnet restore and verify target framework
HTTP 500.31	ASP.NET Core runtime missing	Install correct ASP.NET Core Hosting Bundle
HTTP 500.30	App startup failure	Check appsettings, secrets, DB, logs, Event Viewer
502.5 Process Failure	App crash / hosting issue	Check runtime, hosting bundle, logs, startup exception
CORS blocked	Origin not allowed	Add Angular domain explicitly in CORS policy
Logs not generated	No write permission	Grant Modify only on logs folder

11. Security & Production Hardening

- Use HTTPS only in production
- Do not expose secrets in Angular code
- Do not hardcode production secrets in appsettings when secure configuration is available
- Restrict CORS to trusted origins only
- Use least-privilege folder permissions
- Disable directory browsing in IIS
- Use health check endpoints for monitoring
- Avoid leaving stdoutLogEnabled=true permanently
- Rotate logs and monitor disk usage
- Validate upload paths and file size limits

12. Rollback SOP

Every deployment should have a rollback path. Do not deploy without backup or previous package retention.

Recommended Rollback Strategy

27. Take backup of current Angular deployed folder before replacing files
28. Take backup of current API deployed folder before replacing publish output
29. Retain previous release package with timestamp naming
30. If deployment fails, stop IIS site/app pool (if required)
31. Restore previous known-good Angular build
32. Restore previous known-good API publish package
33. Restart IIS site / app pool
34. Run smoke test immediately
35. Document rollback reason and root cause

Suggested Release Folder Naming

```
D:\Releases\
    ├── AngularApp_2026-03-26_1200\
    ├── MyApi_2026-03-26_1200\
    ├── AngularApp_2026-03-25_1830\
    └── MyApi_2026-03-25_1830\
```

13. Appendices

13.1 Useful Commands

Angular

```
node -v
npm -v
ng version
npm install
ng serve
ng build --configuration production
```

ASP.NET Core

```
dotnet --info
dotnet restore
dotnet build
dotnet run
dotnet publish -c Release -o ./publish
```

IIS

```
iisreset
```

13.2 Naming Standards (Recommended)

- Angular Site Name: <ProjectName>-UI
- API Site Name: <ProjectName>-API

- Angular App Pool: <ProjectName>-UI-Pool
- API App Pool: <ProjectName>-API-Pool
- Release Packages: <ProjectName>_<YYYY-MM-DD>_<HHMM>

13.3 Team Notes / Screenshots Placeholder

Recommended for team version: Insert screenshots here for the following steps:

- IIS feature enablement
- IIS site creation
- Application pool settings
- Folder permissions
- Angular dist folder output
- ASP.NET Core publish folder output
- Swagger validation
- Browser network tab showing successful API call

14. Final Recommendation for Team

For enterprise teams, do not stop at basic deployment. Standardize exact versions, maintain release packages, use health checks, restrict CORS, use environment-specific configuration, and always validate rollback before production deployment. The most common failures are version mismatch, missing Hosting Bundle, incorrect environment URLs, and permission issues.